



Algoritmos e Programação

Desvios Condicionais (if-else) e Operadores Lógicos

Prof. Pedro Bilar Montero
Universidade Federal de Santa Maria
Colégio Politécnico
Curso Técnico em Informática para Internet

Revisão da Aula Anterior

Objetivos da Aula

Desvios Condicionais: if-else

Operadores Lógicos

Prática em Sala

- ▶ **Linguagem C:** por que usar e estrutura mínima de um programa
- ▶ **Tipos de variáveis:** `int`, `float` e `char`
- ▶ **Entrada e saída:** comandos `printf()` e `scanf()`
- ▶ **Fechamento:** exercícios para praticar os conceitos aprendidos

Linguagem C: pontos essenciais

- ▶ C é uma linguagem **compilada**: o código-fonte vira executável
- ▶ É muito usada quando precisamos de **desempenho** e controle
- ▶ Tem uma estrutura clara e simples, ideal para aprender os fundamentos
- ▶ Todo programa parte da função principal: `int main()`
- ▶ Estrutura mínima: `#include <stdio.h>`, `bloco { }, return 0;`

Fluxo geral do programa

Entrada -> Processamento -> Saída

Relembrando: Tipos de Variáveis em C

- ▶ Em C, toda variável precisa ter um **tipo definido**
- ▶ O tipo define quais valores podem ser armazenados
- ▶ O tipo também influencia memória, precisão e operações

Tipo	Uso	Formato
int	Números inteiros	%d
float	Números decimais	%f
char	Um caractere	%c

Regra prática

Escolha o tipo mais adequado ao problema para evitar erros e melhorar a clareza do código.

Relembrando: Exemplos de Variáveis

```
int nota, media;  
int idade = 16;  
float nota = 8.5;
```

Boas práticas

- ▶ Use nomes significativos para facilitar a leitura
- ▶ Evite usar palavras reservadas (como `int`, `float`, `char`)
- ▶ Lembre-se de que o nome da variável não pode conter espaços ou caracteres especiais (exceto `_`)
- ▶ Utilize letras minúsculas e, se necessário, separe palavras com `_` (ex: **nota_final**)



Relembrando: printf() e scanf()

printf() = saída

- ▶ Mostra texto e valores na tela
- ▶ Usa especificadores: %d, %f, %c

```
printf("Idade: %d\n", idade);
```

scanf() = entrada

- ▶ Lê dados digitados pelo usuário
- ▶ Precisa do & antes da variável

```
scanf("%d", &idade);
```

Atenção

Sem & em scanf(), o valor não é gravado corretamente na variável.

Relembrando: Exemplo em C

```
#include <stdio.h>

int main() {
    float saldo, deposito;

    printf("Digite o saldo atual: ");
    scanf("%f", &saldo);

    printf("Digite o valor do depósito: ");
    scanf("%f", &deposito);

    saldo = saldo + deposito;

    printf("Saldo atualizado: %.2f\n", saldo);

    return 0;
}
```

Objetivos da Aula

- ▶ Entender quando usar **desvio condicional** em algoritmos.
- ▶ Escrever decisões com `if`, `if-else` e `if-else if-else`.
- ▶ Aplicar operadores **relacionais** e **lógicos** em C.
- ▶ Resolver problemas com validação de regras e múltiplas condições.

O que é Desvio Condicional?

Ideia central

O programa não precisa seguir sempre o mesmo caminho. Ele pode **decidir** com base em uma condição.

- ▶ Se a condição for verdadeira, executa um bloco.
- ▶ Se for falsa, segue para outro ponto do programa.

```
if (condicao) {  
    // comandos executados se condicao for verdadeira  
} else {  
    // comandos executados se condicao for falsa  
}
```

- ▶ A condição precisa resultar em verdadeiro ou falso.
- ▶ Use chaves, mesmo com um comando, para evitar erros futuros.
- ▶ O bloco else é opcional.
- ▶ O uso de **indentação** ajuda a visualizar os blocos de código.

Exemplo: Aprovação Direta

```
int main() {  
    int nota;  
    printf("Digite a nota: ");  
    scanf("%d", &nota);  
  
    if (nota >= 60) {  
        printf("Aprovado\n");  
    }  
  
    printf("Nota: %d\n", nota);  
  
    return 0;  
}
```

Discussão: O que aparece para o usuário se a nota for 58?

Exemplo: Aprovação Direta

```
int main() {  
    int nota;  
    printf("Digite a nota: ");  
    scanf("%d", &nota);  
  
    if (nota >= 60) {  
        printf("Aprovado\n");  
    } else {  
        printf("Reprovado\n");  
    }  
  
    printf("Nota: %d\n", nota);  
  
    return 0;  
}
```

Discussão: E agora, o que aparece para o usuário se a nota for 58?

Operadores Relacionais

Operador	Significado
>	maior que
<	menor que
>=	maior ou igual
<=	menor ou igual
==	igual a
!=	diferente de

Ponto crítico

== compara valores. **=** faz atribuição.

Operadores Relacionais: Exemplos

```
// Se a idade for maior ou igual a 18, imprima "Maior de idade"
if (idade >= 18) {
    printf("Maior de idade\n");
}
```

```
// Se a letra for 'A', imprima "Turma A"
if (letra == 'A') {
    printf("Turma A\n");
}
```

```
// Se o número for diferente de zero, imprima "Número válido"
if (numero != 0) {
    printf("Numero valido\n");
}
```



Operadores Relacionais: Exemplos

```
// Se a temperatura for menor que 0, imprima "Congelando"
if (temperatura < 0) {
    printf("Congelando\n");
}
```

```
// Se o preço for maior que 100, imprima "Produto caro"
if (preco > 100) {
    printf("Produto caro\n");
}
```

```
// Se a nota for menor ou igual a 5, imprima "Reprovado"
if (nota <= 5) {
    printf("Reprovado\n");
}
```



O que é indentação?

É o recuo (espaço em branco) no início de uma linha de código para mostrar visualmente quais linhas pertencem a cada bloco.

- ▶ Em C, blocos são definidos por { e }.
- ▶ A indentação não muda a execução, mas melhora muito a leitura.
- ▶ Código bem indentado facilita detectar erros de lógica.

Como indentar?

Use tabulações (tab) ou espaços para recuar os comandos dentro de blocos. Mantenha a consistência em todo o código.

Indentação: Exemplo Ruim x Bom

Difícil de ler

```
int main() {  
int nota;  
printf("Digite a nota: ");  
scanf("%d", &nota);  
if (nota >= 60) {  
printf("Aprovado\n");  
} else {  
printf("Reprovado\n");  
}  
return 0;  
}
```

Legível

```
int main() {  
    int nota;  
  
    printf("Digite a nota: ");  
    scanf("%d", &nota);  
  
    if (nota >= 60) {  
        printf("Aprovado\n");  
    } else {  
        printf("Reprovado\n");  
    }  
  
    return 0;  
}
```

Escreva um programa que leia um valor de idade e mostre "Pode dirigir" quando idade for maior ou igual a 18, e "Não pode dirigir" caso contrário.

- ▶ Entrada: idade
- ▶ Saída: "Pode dirigir" ou "Não pode dirigir"
- ▶ Use if-else para cobrir os dois casos
- ▶ Teste com idades 17, 18 e 19 para verificar o resultado

Estrutura Encadeada: if-else if-else

```
if (condicao1) {  
    // bloco 1  
} else if (condicao2) {  
    // bloco 2  
} else {  
    // bloco final  
}
```

- ▶ As condições são testadas de cima para baixo.
- ▶ O primeiro teste verdadeiro encerra a cadeia.

Exemplo: Classificação de Idade

Criar um programa que classifique a idade de uma pessoa em:

- ▶ **Crianca:** idade < 12
- ▶ **Adolescente:** 12 a 17
- ▶ **Adulto:** 18 ou mais

Exemplo: Classificação de Idade

```
int main() {  
    int idade;  
  
    printf("Digite a idade: ");  
    scanf("%d", &idade);  
  
    if (idade < 12) {  
        printf("Crianca\n");  
    } else if (idade < 18) {  
        printf("Adolescente\n");  
    } else {  
        printf("Adulto\n");  
    }  
  
    return 0;  
}
```

Discussão: O que é imprimido se a idade for 12? E se for 18?

Exemplo: Classificação de Idade

```
int main() {  
    int idade;  
  
    printf("Digite a idade: ");  
    scanf("%d", &idade);  
  
    if (idade < 18) {  
        printf("Adolescente\n");  
    } else if (idade < 12) {  
        printf("Crianca\n");  
    } else {  
        printf("Adulto\n");  
    }  
  
    return 0;  
}
```

Discussão: E agora, o que é imprimido se a idade for 12?

Escreva um programa que leia um valor de nota e classifique a situação do aluno:

Regras de classificação:

Nota menor que 5 → **"Reprovado"**

Nota entre 5 e menor que 7 → **"Recuperacao"**

Nota maior ou igual a 7 → **"Aprovado"**

- ▶ Entrada: nota
- ▶ Saída: classificação da nota
- ▶ Use if-else if-else para cobrir os casos
- ▶ Teste com notas 4.5, 5.0, 6.5, 7.0 e 8.0 para verificar o resultado

Por que Operadores Lógicos?

Problema comum

Muitas decisões exigem **mais de uma condição** ao mesmo tempo.

Exemplos:

- ▶ Aprovado se nota ≥ 60 **e** frequência ≥ 75
- ▶ Desconto se cliente é estudante **ou** idoso
- ▶ Acesso permitido se senha correta **e** usuário ativo

Operador && (E lógico)

Regra: só é verdadeiro quando as duas condições são verdadeiras.

```
if (nota >= 60 && frequencia >= 75) {  
    printf("Aprovado\n");  
}
```

Leitura: "nota maior ou igual a 60 e frequência maior ou igual a 75".

Regra: é verdadeiro quando pelo menos uma condição é verdadeira.

```
if (idade < 16 || temAutorizacao == 1) {  
    printf("Pode entrar na atividade\n");  
}
```

Leitura: basta uma das condições para permitir a entrada. Se a idade for menor que 16 **ou** se tiver autorização, o acesso é permitido.

Operador ! (NÃO lógico)

Regra: inverte o resultado de uma condição.

```
if (!(idade >= 18)) {  
    printf("Acesso nao permitido para menores de idade\n");  
}
```

Leitura: se a idade **não for maior ou igual a 18**, o bloco executa.

Tabela Verdade dos Operadores Lógicos

A	B	A && B	A B
V	V	V	V
V	F	F	V
F	V	F	V
F	F	F	F

A	!A
V	F
F	V

Leitura rápida

&& (E): só dá verdadeiro quando as duas condições são verdadeiras.

|| (OU): dá verdadeiro quando pelo menos uma condição é verdadeira.

Operador NOT

! inverte o valor lógico.

Boa prática

Mesmo sabendo a precedência, use parênteses para deixar a intenção clara.

```
if ((nota >= 60 && frequencia >= 75) || recuperacao == 1) {  
    printf("Aprovado\n");  
}
```

Vantagem: facilita leitura, revisão e manutenção.

1) Confundir atribuição e comparação

```
if (nota = 10) { ... }    // errado  
if (nota == 10) { ... }  // correto
```

2) Esquecer limites

```
if (idade > 18) { ... }   // exclui idade 18  
if (idade >= 18) { ... }  // inclui idade 18
```

Exemplo: Aprovação Escolar

```
int nota, frequencia;  
scanf("%d", &nota);  
scanf("%d", &frequencia);  
  
if (nota >= 60 && frequencia >= 75) {  
    printf("Aprovado\n");  
} else {  
    printf("Reprovado\n");  
}
```

Exemplo: Acesso ao Sistema

```
int senhaCorreta, matricula = 0;
scanf("%d", &senhaCorreta);
scanf("%d", &matricula);

if (senhaCorreta == 1 || matricula > 0) {
    printf("Acesso liberado\n");
} else {
    printf("Acesso negado\n");
}
```

Exemplo: Desconto na Loja

```
float valorCompra;  
int clienteVip; // 1 para cliente VIP, 0 para cliente comum  
  
printf("Digite o valor da compra: ");  
scanf("%f", &valorCompra);  
printf("O cliente é VIP? (1 para sim, 0 para não): ");  
scanf("%d", &clienteVip);  
  
if (valorCompra >= 200.0 || clienteVip == 1) {  
    printf("Desconto aplicado\n");  
} else {  
    printf("Sem desconto\n");  
}
```

Exercício 1 : Número Par ou Ímpar

Tarefa

- ▶ Ler um número inteiro e determinar se é par ou ímpar.
- ▶ Use o operador % para verificar o resto da divisão por 2.
- ▶ Imprima "Par" se o número for par, e "Ímpar" caso contrário.
- ▶ Teste com números como 4, 7, 0 e -3 para verificar o resultado.

Critérios técnicos

- ▶ Usar if-else.
- ▶ Considerar idade 18 como maior de idade.

Tempo: 6 minutos.

Exercício 2: Nota e Frequência

Tarefa

- ▶ Ler nota e frequência.
- ▶ Aprovar se `nota >= 60 e frequencia >= 75`.
- ▶ Imprimir "Aprovado" ou "Reprovado".
- ▶ Testar com casos como (60, 75), (59, 80), (70, 74) e (80, 90).

Tempo: 10 minutos.

Exercício 3: Menor de três números

Tarefa

- ▶ Ler três números inteiros.
- ▶ Determinar qual é o menor entre eles.
- ▶ Imprimir o menor número encontrado.
- ▶ Testar com casos como (5, 3, 8), (10, 10, 5) e (-1, -5, -3).

Tempo: 10 minutos.

- ▶ **Compreendemos:** como usar decisões com `if`, `if-else` e `if-else if-else`
- ▶ **Aplicamos:** operadores relacionais (`>`, `<`, `>=`, `<=`, `==`, `!=`)
- ▶ **Combinamos:** condições com operadores lógicos (`&&`, `||`, `!`)
- ▶ **Praticamos:** resolução de problemas com regras reais e testes de casos
- ▶ **Ponto-chave:** escrever condições claras evita erros de lógica

Próxima aula

Lista prática com foco em validação de entradas e decisões encadeadas.